

Project #8 — Dominator Problem

Divide & Conquer vs Insertion Sort

Algorithm 1 — Divide & Conquer (Recursive)

Idea: Split the array in half recursively until each part has one element. Merge back up by checking which candidate is dominant in the combined range.

Pseudo-Code

Divide & Conquer Dominator — Pseudo-Code

```
FUNCTION dominatorDC(A[], lowestIndex, highestIndex):

    // Base case: single element
    IF lowestIndex = highestIndex THEN RETURN A[lowestIndex]

    mid <- (lowestIndex + highestIndex) / 2

    // Divide
    leftCand <- dominatorDC(A, lowestIndex, mid)
    rightCand <- dominatorDC(A, mid+1, highestIndex)

    // Merge: trivial case
    IF leftCand = rightCand THEN RETURN leftCand

    size <- highestIndex - lowestIndex + 1

    // Count left candidate in full range
    IF leftCand != NO_CANDIDATE THEN
        lCount <- COUNT(A, lowestIndex, highestIndex, leftCand)
        IF lCount > size/2 THEN RETURN leftCand
    END IF

    // Count right candidate in full range
    IF rightCand != NO_CANDIDATE THEN
        rCount <- COUNT(A, lowestIndex, highestIndex, rightCand)
        IF rCount > size/2 THEN RETURN rightCand
    END IF

    RETURN NO_CANDIDATE
END FUNCTION

FUNCTION COUNT(A[], lo, hi, val):
    freq <- 0
    FOR i <- lo TO hi DO
        IF A[i] = val THEN freq <- freq + 1
    RETURN freq
END FUNCTION

FUNCTION findDominator_DC(A[0..n-1]):
    candidate <- dominatorDC(A, 0, n-1)
    IF candidate = NO_CANDIDATE THEN RETURN empty
    FOR i <- 0 TO n-1 DO
        IF A[i] = candidate THEN add i to indices
    RETURN indices
END FUNCTION
```

Complexity Analysis

Recurrence Relation:

Recurrence Relation — Algorithm 1

$$T(n) = 2 * T(n/2) + n$$

Because:

- We split into 2 subproblems of size $n/2$
- COUNT() does n work at each merge level

Solving with Master Theorem:

Master Theorem Solution

$$T(n) = a * T(n/b) + f(n)$$

$a = 2$ (two recursive calls)

$b = 2$ (array split in half)

$f(n) = n$

$$\log_b(a) = \log_2(2) = 1$$

$f(n) = n = O(n^1) \rightarrow$ Case 2 of Master Theorem

Result: $T(n) = O(n \log n)$

Space: recursion depth = $\log_2(n) \rightarrow O(\log n)$

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$

Best Case: $O(n \log n)$ | **Worst Case:** $O(n \log n)$ | **Average:** $O(n \log n)$

Algorithm 2 — Insertion Sort (Non-Recursive)

Idea: Sort a copy of the array using Insertion Sort, then check if the middle element is a majority by comparing $A[i]$ with $A[i + n/2]$. If equal, that value is the Dominator.

Pseudo-Code

Insertion Sort Dominator — Pseudo-Code

```
FUNCTION insertionSort(arr[]):
  N <- length(arr)
  FOR i <- 1 TO N-1 DO
    key <- arr[i]
    j <- i - 1
    WHILE j >= 0 AND arr[j] > key DO
      arr[j+1] <- arr[j]
      j <- j - 1
    END WHILE
    arr[j+1] <- key
  END FOR
END FUNCTION

FUNCTION printDominatorIndices(original[], length):
  IF length = 0 THEN RETURN NO_DOMINATOR
  IF length = 1 THEN RETURN index 0

  sorted <- COPY of original
  CALL insertionSort(sorted)

  half <- length / 2
  candidate <- NOT_FOUND

  FOR i <- 0 TO length - half - 1 DO
    IF sorted[i] = sorted[i + half] THEN
      candidate <- sorted[i]
      BREAK
    END IF
  END FOR

  IF candidate = NOT_FOUND THEN RETURN NO_DOMINATOR

  FOR i <- 0 TO length-1 DO
    IF original[i] = candidate THEN PRINT i
  END FOR

  RETURN candidate
END FUNCTION
```

Complexity Analysis

Step-by-step count:

Phase	Operations	Explanation
Insertion Sort	$O(n^2)$	Each element may shift through all sorted elements
Copy array	n	One pass to copy original into sorted
Find candidate	$\leq n$	One pass over sorted array to find majority
Print indices	n	One pass over original array
Total	$O(n^2)$	Dominated by Insertion Sort step

Recurrence Relation:

Recurrence Relation — Algorithm 2

$$T(n) = T(n-1) + n \quad (\text{Insertion Sort})$$
$$T(1) = 1$$

Solving:

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &= T(1) + 2 + 3 + \dots + n \\ &= 1 + n(n+1)/2 - 1 \\ &= n(n+1)/2 \\ &= O(n^2) \end{aligned}$$

Time Complexity: $O(n^2)$
Space Complexity: $O(n)$
Best Case: $O(n)$ | **Worst Case:** $O(n^2)$ | **Average:** $O(n^2)$

Comparison

Criterion	Algorithm 1 — Divide & Conquer	Algorithm 2 — Insertion Sort
Paradigm	Recursive Splitting	Sort-based Linear Scan
Time Complexity	$O(n \log n)$	$O(n^2)$
Space Complexity	$O(\log n)$ stack	$O(n)$
Approach	Divide & Conquer	Insertion Sort + Search
Code complexity	Moderate	Simple
Returns indices	All occurrences	All occurrences
Preferred	Yes (faster)	No (slower)

Conclusion:

Divide & Conquer is the faster choice at $O(n \log n)$ time and $O(\log n)$ space.

Insertion Sort is simpler to understand but slower at $O(n^2)$ due to the sorting step.